



INSTITUTO POLITÉCNICO NACIONAL

UNIDAD PROFESIONAL INTERDISCIPLINARIA DE INGENIERÍA Y CIENCIAS SOCIALES Y ADMINISTRATIVAS

MATERIA: ABSTRACCIÓN Y USO DE DATOS

Reporte de prácticas 2do Parcial

GRUPO: 3CM30

PROFESOR: Yventz Entzana Garduño

ALUMNOS: Cruz Ortega Omar Josue

Mondragon Ortiz Itzel Andre

No. Lista

5

27

Contenido

Enlace: Repositorio de GitHub	4
Prácticas de Ordenamiento (16 a la 18)	4
P16-Análisis del Algoritmo de Ordenamiento de Burbuja en POO	4
P17-Análisis del Algoritmo Merge Sort aplicado a Estructuras de Datos	5
P18-Implementación algorítmica de Quick Sort en C++	7
Prácticas de Ordenamiento indirecto (19 a la 21)	10
P19- Gestión de Abordaje y Análisis hacia el Ordenamiento de Burbuja Indirecto 10	
P20- Optimización de Memoria mediante Merge Sort Indirecto	11
P21- Optimización de Memoria con Quick Sort Indirecto	13
Punteros para el manejo indirecto con clases y estructuras (P1)	16
ADT	18
Arreglo - Pila, Cola y Lista (P3 a la P5).....	18
P3 - Análisis Lógico de la Pila (Stack)	18
P4- Análisis Lógico de la Cola (Queue)	19
P5- Análisis Lógico de la Lista	20
Puntero - Pila, Cola y Lista (P6 a la P7Bis)	21
P6- Lógica y Sintaxis de la Pila	21
P7- Lógica y Sintaxis de la Cola.....	23
P7BIS - Lógica y Sintaxis de la Lista.....	24
Librerías - Pila, Cola y Lista (P8 a la P10)	26
P8 - Lógica y Sintaxis de la Pila	26
P9 - Lógica y Sintaxis de la Cola	27
P10 - Lógica y Sintaxis de la Lista	28
ADT - Nuevo tipo de dato	29
Arreglo - Pila, Cola y Lista (P3 a la P5).....	29
P3.1-Implementación de Pila (Stack)	29
P4.1 -Implementación de Cola	30
P5.1 -Implementación de Lista	31
Puntero – Pila, Cola y Lista (P6 a la P7Bis)	32

P6.1 -Implementación de Pila.....	32
P7.1 -Implementación de Cola.....	33
P7.1Bis -Implementación de Lista.....	34
Librerías – Pila, Cola y Lista (P8 a la P10)	35
P8.1 -Implementación de Pila	35
P9.1 -Implementación de Cola	37
P10.1 -Implementación de Lista	38

Enlace: Repositorio de GitHub

Enlace al repositorio : [OmarC777/Practicas-ORD](https://github.com/OmarC777/Practicas-ORD)

Prácticas de Ordenamiento (16 a la 18)

P16-Análisis del Algoritmo de Ordenamiento de Burbuja en POO

1. Introducción al Problema de Ordenamiento

Uno de los requerimientos principales fue la capacidad de presentar la información de manera organizada. Para lograr esto, implementamos el clásico algoritmo de ordenamiento de burbuja (Bubble Sort) adaptado al paradigma de Programación Orientada a Objetos.

2. Definición e Implementación del Algoritmo

La lógica de ordenamiento fue encapsulada dentro de la clase Alumnos. En nuestro archivo de cabecera Alumnos.h, declaramos el método burOb, el cual está diseñado para recibir como parámetros un arreglo completo de tipo Alumnos y un entero n que representa la cantidad de elementos registrados.

La implementación real del algoritmo funciona de la siguiente manera:

- Ciclos Anidados: El método utiliza dos bucles for anidados para recorrer el arreglo. El ciclo externo (i) controla el número de pasadas necesarias, mientras que el ciclo interno (j) se encarga de recorrer los elementos no ordenados.
- Criterio de Evaluación: En cada iteración del ciclo interno, se realiza una comparación adyacente utilizando la condición `if (grupo[j].getPromedio() > grupo[j + 1].getPromedio())`. Aquí es donde la POO brilla, ya que usamos el método getter `getPromedio()` para acceder de forma segura al atributo que define nuestro criterio de ordenamiento.
- Criterio de Evaluación: En cada iteración del ciclo interno, se realiza una comparación adyacente utilizando la condición `if (grupo[j].getPromedio() > grupo[j + 1].getPromedio())`. Aquí es donde la POO brilla, ya que usamos el método getter `getPromedio()` para acceder de forma segura al atributo que define nuestro criterio de ordenamiento.
- Intercambio (Swap): Si el promedio del alumno en la posición actual es mayor que el del siguiente, procedemos a intercambiar sus posiciones en la memoria del arreglo. Para no perder la información durante el cambio,

instanciamos un objeto auxiliar temporal (`Alumnos temp = grupo[j];`) que guarda el registro completo del alumno, no solo su promedio.

Adicionalmente, en `Alumnos.h` definimos una segunda versión del método llamada `burChar`, diseñada para ordenar arreglos de caracteres. Esta versión en `Alumnos.cpp` replica la misma lógica algorítmica de los ciclos y el intercambio, pero utilizando una variable temporal de tipo `char`, lo que nos sirve como demostración de cómo el mismo principio lógico se aplica a distintos tipos de datos.

3. Integración en el Flujo del Programa principal

El algoritmo entra en acción dentro de nuestro archivo `main.cpp` cuando el usuario selecciona la opción 3 ("Mostrar registros").

Antes de iterar el arreglo para imprimirlo, el programa invoca el ordenamiento mediante la instrucción `lista[0].burOb(lista, cantidadRegistrada);`. De esta manera, garantizamos que el usuario siempre vea la información clasificada de menor a mayor promedio de forma dinámica, independientemente del orden en que los alumnos fueron ingresados originalmente.

4. Conclusiones y Observaciones Críticas

Desde una perspectiva de Estructuras de Datos, el ordenamiento de burbuja tiene una complejidad temporal de $O(n^2)$, lo que significa que el tiempo de ejecución crece cuadráticamente. Dado que nuestro programa en `main.cpp` tiene un límite constante `MAX = 10`, el impacto en el rendimiento es imperceptible y el algoritmo cumple su función a la perfección gracias a su sencillez de implementación.

Sin embargo, como observación crítica al diseño de nuestro código, notamos una particularidad en la llamada `lista[0].burOb(...)`. Estamos utilizando una instancia específica para ordenar todo el grupo de alumnos.

P17-Análisis del Algoritmo Merge Sort aplicado a Estructuras de Datos

1. Introducción

En esta práctica de programación en C++, avanzamos en el estudio de algoritmos de ordenamiento al implementar el método Merge Sort (Ordenamiento por Mezcla). A diferencia de los algoritmos de ordenamiento de nivel básico con complejidad $O(n^2)$ que utilizábamos en semestres anteriores, Merge Sort emplea el paradigma de "divide y vencerás". El objetivo de este reporte es documentar cómo se estructuró este algoritmo para gestionar y ordenar un Tipo de Dato Abstracto (TDA) que simula los registros de un grupo de empleados.

2. Arquitectura de Clases y Modularidad

Para mantener un código limpio y modular, dividimos el programa separando la estructura de los datos de la lógica matemática.

- Clase Empleado: Actúa como nuestro TDA base, almacenando los atributos públicos id y nombre.
- Clase Ordenamiento: Encapsula exclusivamente los métodos necesarios para ejecutar el Merge Sort. Esta clase presenta polimorfismo paramétrico (sobrecarga conceptual), ya que cuenta con métodos para ordenar tanto arreglos de enteros primitivos (Enteros, marEnteros) como arreglos de objetos (Empleados, marEmpleados).

3. Análisis Profundo del Algoritmo Merge Sort

El núcleo algorítmico de esta práctica reside en el archivo Empleado.cpp. La implementación del Merge Sort requiere dos funciones principales que trabajan en conjunto a través de la recursividad y la gestión de memoria dinámica:

A. La Fase de División (marEmpleados)

Esta función es la encargada de dividir recursivamente el problema. Recibe el arreglo original y los índices de los límites (ini y fin).

- Su caso base o condición de paro es `if (ini >= fin)`, lo que significa que el subarreglo tiene un solo elemento y, por definición, ya está ordenado.
- Si la condición no se cumple, calcula el índice central (med) y realiza dos llamadas recursivas a sí misma: una para la mitad izquierda (`ini` a `med`) y otra para la mitad derecha (`med + 1` a `fin`).
- Una vez que las divisiones alcanzan su nivel más atómico, se invoca a la función de mezcla Empleados.
-

B. La Fase de Conquista y Mezcla (Empleados)

Esta es la función más compleja computacionalmente, responsable de fusionar dos subarreglos previamente ordenados.

- Memoria Dinámica (Punteros): Aquí ponemos en práctica el uso de la memoria dinámica en el heap. Calculamos el tamaño de los dos subarreglos (`n1` y `n2`) y utilizamos el operador `new` para instanciar dos arreglos temporales de punteros: `Empleado* izq = new Empleado[n1];` y `Empleado* der = new Empleado[n2];`.
- Criterio de Evaluación: Mediante un ciclo `while`, comparamos los elementos de ambos arreglos temporales basándonos en nuestro atributo clave: `if`

(`izq[i].id <= der[j].id`). El objeto con el ID menor se inserta en el arreglo original `arre[k]`.

- Vaciado de Remanentes: Se utilizan dos ciclos `while` adicionales para asegurar que, si un subarreglo se agota antes que el otro, los elementos restantes se copien directamente al arreglo final.
- Prevención de Fugas de Memoria (Memory Leaks): Para garantizar un sistema estable y aplicar las buenas prácticas de la carrera, al final de la función liberamos la memoria dinámica que solicitamos utilizando el operador `delete[]` `izq`; y `delete[]` `der`;

4. Integración en el Flujo Principal

En la aplicación final `main.cpp`, gestionamos un arreglo estático de tamaño fijo (`const int MAX = 10`). La potencia del Merge Sort se demuestra en la opción 3 del menú ("Mostrar registros").

Instanciamos un objeto `obMarge` de la clase `Ordenamiento` y, antes de imprimir la lista de empleados, la ordenamos llamando al método principal: `obMarge.marEmpleados(lista, 0, cantidadRegistrada - 1)`. Pasamos el índice 0 como inicio y el índice del último elemento ingresado como fin, garantizando un ordenamiento exacto sin procesar los espacios vacíos del arreglo estático.

5. Conclusiones Técnicas

Avanzar hacia algoritmos como Merge Sort marca una pauta importante en nuestro desarrollo como programadores. Aunque su implementación es mucho más verbosa que algoritmos elementales, su complejidad temporal de $O(n \log n)$ garantiza un rendimiento óptimo y consistente incluso en el peor de los casos. Además, esta práctica nos obligó a salir de la zona de confort de los arreglos puramente estáticos, combinando las listas estáticas de `main.cpp` con la instanciación de arreglos dinámicos a través de punteros durante el proceso de ordenamiento de los objetos.

P18-Implementación algorítmica de Quick Sort en C++

1. Introducción

En esta práctica de programación, implementamos el algoritmo de ordenamiento Quick Sort (Ordenamiento Rápido). Al igual que Merge Sort, este algoritmo utiliza el paradigma de "divide y vencerás" mediante recursividad, pero con una ventaja crucial: ordena los elementos in-place (en su lugar), lo que significa que no requiere solicitar memoria adicional (dinámica) para hacer las mezclas. El objetivo de este

reporte es analizar la lógica matemática detrás de este algoritmo al ordenar un Tipo de Dato Abstracto (TDA) simulando registros de empleados.

2. Estructura y Diseño Orientado a Objetos

Para este programa, estructuramos el código separando las responsabilidades de los objetos de la lógica de ordenamiento. En nuestro archivo de encabezado (Empleado.h), definimos dos clases independientes:

- Clase Empleado: Funciona como nuestro contenedor de datos con los atributos públicos id y nombre.
- Clase OrdenadorQS: Es una clase de utilidad exclusiva para encapsular los algoritmos de ordenamiento. Para demostrar la versatilidad de la lógica, diseñamos la clase para que sea capaz de ordenar arreglos de enteros primitivos, caracteres y objetos Empleado mediante la definición de métodos específicos para cada tipo de dato.

3. Análisis de la Lógica de Quick Sort

El corazón de esta práctica se encuentra en la implementación matemática dentro del archivo de origen (Empleado.cpp). El ordenamiento de nuestros objetos Empleado se logra mediante la interacción de dos métodos funda

A. El Método de Partición (particionEmpleados)

La eficiencia de Quick Sort depende completamente de cómo se elija y posicione el "pivote". Este método recibe el arreglo, el índice inicial (ini) y el índice final (fin).

- Selección del Pivote: En nuestra implementación, establecimos por diseño que el pivote siempre será el ID del último elemento del subarreglo actual: `int pivote = arre[fin].id;`
- Recorrido e Intercambio: Utilizamos una variable `i` (iniciada en `ini - 1`) para rastrear el límite de los elementos menores al pivote. Con un ciclo `for`, iteramos con la variable `j` desde el inicio hasta un elemento antes del fin (`fin - 1`).
- Criterio de Evaluación: Si el ID del elemento actual es menor o igual al pivote (`if (arre[j].id <= pivote)`), incrementamos `i` y realizamos un intercambio (`swap`) utilizando una variable temporal `Empleado temp` para no perder los datos del objeto.
- Posicionamiento Final: Una vez que termina el ciclo, sabemos que todos los elementos menores están a la izquierda de `i`. Por lo tanto, movemos el pivote a su posición final definitiva intercambiando `arre[fin]` con `arre[i + 1]`. El método retorna este índice, el cual divide al arreglo en dos mitades.

B. El Método Recursivo (ordenarEmpleados)

Este es el método controlador. Su caso base (condición de paro) es evaluar que el índice inicial sea menor al final (if (ini < fin)), lo que nos asegura que el subarreglo tiene al menos dos elementos para ordenar.

- Llama al método `particionEmpleados` para obtener el `indicePivote` exacto.
- Realiza dos llamadas recursivas a sí mismo: la primera ordena exclusivamente la sublista izquierda (ini hasta `indicePivote - 1`) y la segunda ordena la sublista derecha (`indicePivote`

4. Implementación en el Programa Principal

En nuestra aplicación (main.cpp), utilizamos un arreglo estático con una constante de capacidad máxima de 10 empleados (`const int MAX = 10;`).

La invocación del algoritmo ocurre en la opción 3 del menú interactivo. Antes de imprimir, instanciamos nuestro objeto `obQuick` de la clase `OrdenadorQS` y llamamos al método: `obQuick.ordenarEmpleados(lista, 0, cantidadRegistrada - 1);`. Es vital notar que como límite final pasamos `cantidadRegistrada - 1` y no la constante `MAX`. Esto optimiza el proceso al máximo, evitando que Quick Sort intente procesar e intercambiar celdas de memoria vacías o con datos basura.

5. Conclusiones

Como estudiantes de Ciencias de la Informática, comprender Quick Sort es fundamental para la optimización de recursos. Aunque su complejidad en el peor de los casos (cuando la lista ya está ordenada al revés) puede caer a $O(n^2)$, su rendimiento promedio de $O(n \log n)$ y su arquitectura de ordenamiento in-place lo hacen superior a Merge Sort en entornos donde la memoria es limitada, ya que nos ahorramos por completo el uso de punteros y la instrucción `new` para crear arreglos temporales.

Prácticas de Ordenamiento indirecto (19 a la 21)

P19- Gestión de Abordaje y Análisis hacia el Ordenamiento de Burbuja Indirecto

1. Introducción al Sistema

En esta práctica desarrollamos un sistema. El núcleo de nuestra aplicación en la función principal maneja un arreglo estático definido con una constante $MAX = 20$. A través de un menú interactivo, el programa permite el ingreso de registros, la eliminación mediante desplazamientos lógicos (identificando el número de asiento), y la organización de los datos.

2. Análisis de la Implementación Algorítmica Actual

Para mantener buenas prácticas de modularidad, la arquitectura del programa separa la estructura de información de la lógica matemática. Tenemos la clase Pasajeros, que define los atributos de numeroAsiento y nombre. Por otro lado, la clase Ordenamiento concentra los algoritmos de clasificación.

En la versión actual del código, el método burbujaPasajeros implementa un Bubble Sort clásico y directo:

El algoritmo realiza ciclos anidados para comparar el atributo numeroAsiento de cada pasajero.

Cuando encuentra un asiento mayor que el adyacente, ejecuta un intercambio (swap) instanciando un objeto completo temporal: `Pasajeros temp = arreglo[j];`.

Posteriormente, sobrescribe físicamente las posiciones de memoria reasignando los objetos completos.

Adicionalmente, el código demuestra polimorfismo mediante la sobrecarga conceptual, ya que la clase puede ordenar tanto enteros numéricos como arreglos de caracteres, características que el usuario puede probar independientemente desde la opción 1 del menú.

3. El Problema del Costo Computacional

Aunque nuestra implementación directa funciona sin problemas para un límite pequeño de 20 personas, como estudiantes de Ciencias de la Informática debemos proyectar estos sistemas a escenarios reales. Intercambiar registros usando `Pasajeros temp` exige que el procesador copie y mueva cada atributo de la clase (enteros y cadenas de texto). Si en el futuro nuestra clase escalara para incluir el

peso del equipaje, la clase de boleto, el destino y el pasaporte, el tiempo de ejecución aumentaría drásticamente debido a la saturación del bus de datos copiando información pesada.

4. Propuesta de Refactorización: Ordenamiento de Burbuja Indirecto

Aquí es donde surge la necesidad de aplicar el Ordenamiento de Burbuja Indirecto. Dado que en estas fechas de evaluaciones hemos estado repasando a fondo la recursividad y el comportamiento avanzado de los punteros, podemos aplicar estos conceptos para optimizar el método de intercambio.

En lugar de alterar el arreglo original, el diseño lógico de un ordenamiento indirecto funcionaría de la siguiente manera:

- Arreglo de Direcciones: Declararíamos un arreglo paralelo de punteros (ej. Pasajeros* indices[MAX]), donde cada elemento apunte directamente a las posiciones estáticas de los pasajeros originales.
- Comparación por Referencia: El ciclo de burbuja iteraría sobre el arreglo de índices, comparando los valores a través del operador flecha: if (indices[j]->numeroAsiento > indices[j+1]->numeroAsiento).
- Intercambio Ligero: Al momento de hacer el swap, solo intercambiaríamos las direcciones de memoria que almacenan los punteros, jamás los objetos Pasajeros.
- Presentación de Datos: En la opción 5 de nuestro menú (Ordenar y Mostrar Pasajeros), el bucle for recorrería el arreglo de punteros, permitiéndonos imprimir la lista en orden numérico sin haber gastado recursos en mover los datos reales.

5. Conclusiones

La estructura de este programa cumple exitosamente con organizar la información mediante comparaciones iterativas. Sin embargo, proyectar nuestro análisis hacia un ordenamiento indirecto refleja una comprensión más profunda de la arquitectura de la memoria en C++. Al dominar el uso de arreglos paralelos y punteros, garantizamos que nuestro software será capaz de clasificar estructuras de datos robustas y pesadas de forma casi instantánea, un requisito indispensable para el desarrollo de sistemas de información profesionales.

P20- Optimización de Memoria mediante Merge Sort Indirecto

1. Introducción

Como estudiantes de la carrera de Ciencias de la Informática, uno de nuestros objetivos fundamentales al cursar Estructuras de Datos es garantizar la eficiencia

algorítmica y el uso correcto de la memoria. En esta práctica implementamos el algoritmo Merge Sort, pero integrando una técnica de optimización avanzada: el ordenamiento indirecto. El objetivo de este reporte es analizar cómo este enfoque nos permite ordenar registros lógicamente, como los de un empleado, sin necesidad de mover físicamente sus datos en la memoria RAM, reduciendo drásticamente el costo computacional.

2. Arquitectura del Proyecto y Firmas de Funciones

Para soportar el ordenamiento indirecto, tuvimos que modificar la estructura de nuestras clases en el archivo de cabecera Empleado.h.

Conservamos el Tipo de Dato Abstracto Empleado con sus propiedades fundamentales: id y nombre.

La modificación clave ocurrió en la clase Ordenamiento. Ahora, las firmas de los métodos `marEnteros`, `marCaracteres` y `marEmpleados` (junto con sus funciones de mezcla) obligan a recibir por parámetro no solo el arreglo original de los datos, sino un arreglo adicional llamado `int indices[]`.

3. Lógica Algorítmica: La Mezcla Indirecta

El núcleo matemático de este enfoque se encuentra en el archivo Empleado.cpp. Al ejecutar la función recursiva y llegar a la fase de conquista (el método `Empleados`), el algoritmo altera su comportamiento tradicional:

Uso de Memoria Dinámica: Para realizar la mezcla, solicitamos memoria en el heap utilizando el operador `new` para crear dos arreglos temporales de sub-índices (`izq` y `der`). Llenamos estos arreglos temporales copiando exclusivamente los números del arreglo `indices`, no los objetos `Empleado`.

Comparación Cruzada: La genialidad del ordenamiento indirecto ocurre en la condición `if (arre[izq[i]].id <= arre[der[j]].id)`. Aquí evaluamos el ID de los empleados originales a través del "mapa" que nos proporcionan los índices.

Intercambio Ligero: Al momento de ordenar, la instrucción ejecutada es `indices[k] = izq[i];` o `indices[k] = der[j];`. Jamás reasignamos los objetos `Empleado` completos. Simplemente modificamos el arreglo de números enteros que nos indica en qué posición virtual debería ir cada registro.

Para evitar fugas de memoria, al terminar la mezcla liberamos la memoria de nuestros arreglos temporales con `delete[] izq;` y `delete[] der;`.

4. Implementación y Gestión de la Interfaz Principal

En nuestra función principal dentro del código de aplicación, pusimos en marcha este concepto gestionando la memoria con punteros.

Al iniciar, instanciamos tanto nuestra lista principal (`Empleado* listaEmpleados = new Empleado[capacidadMaxima];`) como nuestro mapa paralelo (`int* indicesEmpleados = new int[capacidadMaxima];`).

- Cuando el usuario selecciona la opción 5 ("Ordenar y Mostrar Empleados"), el programa realiza un paso previo crítico: inicializa el arreglo de índices (`indicesEmpleados[i] = i`) para que actúe como un reflejo 1 a 1 de la lista desordenada actual.

Tras ejecutar `obMarge.marEmpleados`, mandamos llamar a la función utilitaria `imprimirEmpleados`. Esta función itera sobre nuestra estructura ejecutando `cout << arre[indices[i]].id;`. Así, la pantalla muestra los datos de menor a mayor, aunque físicamente el arreglo `listaEmpleados` sigue estando en el mismo desorden con el que el usuario capturó los datos.

5. Conclusiones y Observaciones

La implementación del Merge Sort Indirecto es un salto de calidad en nuestra formación en Ciencias de la Informática. Si nuestra clase `Empleado` escalara en el futuro para contener fotografías, descripciones largas o historiales médicos pesados, un ordenamiento tradicional colapsaría el rendimiento al estar moviendo todos esos bytes de un lado a otro. El enfoque indirecto resuelve este problema ordenando únicamente arreglos de enteros (índices), garantizando que el rendimiento de nuestra aplicación se mantenga rápido y estable, logrando esa preciada complejidad temporal de $O(n \log n)$.

Como única área de mejora detectada, notamos que en la opción 3 (Quitar Empleado), el programa sigue utilizando un reacomodo de memoria directo, desplazando los objetos físicamente mediante un ciclo `for` (`listaEmpleados[j] = listaEmpleados[j + 1];`). Para futuras versiones del sistema, podríamos buscar implementar listas ligadas o un borrado lógico absoluto para mantener el paradigma indirecto en todo el ciclo de vida del programa.

P21- Optimización de Memoria con Quick Sort Indirecto

1. Introducción

En esta práctica de la asignatura de Estructuras de Datos, implementamos un sistema de gestión para los registros de empleados. El núcleo de este proyecto no es solo ordenar la información, sino hacerlo de la forma más eficiente posible para el procesador y la memoria RAM. Para lograrlo, implementamos el algoritmo Quick

Sort utilizando la técnica de ordenamiento indirecto. El objetivo de este reporte es analizar cómo esta técnica nos permite clasificar arreglos de objetos complejos sin el costo computacional de moverlos físicamente de sus direcciones de memoria originales.

2. Estructura del Proyecto y Modificación de Firmas

Para aplicar el ordenamiento indirecto, tuvimos que replantear la forma en que nuestras clases se comunican en el archivo de cabecera Empleado.h.

Mantenemos nuestra clase base Empleado, la cual funciona como nuestro Tipo de Dato Abstracto (TDA) almacenando el id y el nombre.

Creamos la clase controladora OrdenadorQS. La diferencia fundamental respecto a algoritmos directos es que agregamos un parámetro extra a todas las funciones de ordenamiento y partición: un arreglo `int indices[]`. Esta estructura es capaz de ordenar arreglos de enteros, caracteres y objetos Empleado mediante sobrecarga conceptual.

3. Análisis Algorítmico: La Magia de la Partición Indirecta

El corazón del algoritmo Quick Sort reside en su función de partición. Al analizar el archivo Empleado.cpp, podemos observar cómo la lógica tradicional de "divide y vencerás" se adapta para no tocar los datos originales:

A. Selección y Comparación a través del "Espejo"

En el método `particionEmpleados`, no tomamos el pivote directamente del arreglo de objetos. En su lugar, obtenemos el ID del pivote basándonos en el arreglo de índices: `int pivote = arre[indices[fin]].id;`

De igual forma, cuando el bucle `for` recorre los elementos, la evaluación `if (arre[indices[j]].id <= pivote)` no compara posiciones contiguas reales en la memoria del arreglo `arre`, sino que "salta" a las posiciones que el arreglo de índices le dicta en ese momento.

B. El Intercambio Ligero (Swap)

Aquí es donde aplicamos los conceptos de optimización que hemos visto en el semestre. Si la condición del `if` se cumple, necesitamos hacer un intercambio. En un ordenamiento tradicional, esto implicaría copiar un objeto Empleado completo (incluyendo su cadena de texto nombre) a una variable temporal. En nuestra implementación, el intercambio es sumamente ligero como indica la documentación de nuestro código, solo movemos variables primitivas de tipo `int` (los índices). Los

objetos Empleado jamás se mueven de la posición en la que el usuario los capturó originalmente.

C. La Recursividad (ordenarEmpleados)

El método controlador sigue la arquitectura estándar de Quick Sort. Si el índice inicial es menor al final ($ini < fin$), calcula el índicePivote llamando a la partición y luego se invoca a sí mismo recursivamente dos veces: una para la sublista izquierda (ini a índicePivote - 1) y otra para la sublista derecha (índicePivote + 1 a fin).

4. Implementación en la Interfaz de Usuario

Para que este concepto abstracto funcione en nuestra aplicación (main.cpp), gestionamos arreglos paralelos estáticos. El programa permite un máximo de 20 empleados mediante la constante `MAX_EMPLEADOS = 20`.

El proceso completo se materializa en la opción 5 del menú ("Ordenar y Mostrar Empleados"):

Inicialización: Antes de llamar a Quick Sort, preparamos nuestro arreglo paralelo creando un ciclo for que asigna a cada elemento su propio índice (`indEmpleados[i] = i;`).

Ejecución: Llamamos a `obQuick.ordenarEmpleados`, pasándole tanto la lista real como la lista de índices.

1. Visualización: Al imprimir, utilizamos un ciclo que evalúa `empleadosRegistrados[indEmpleados[i]]`. Esto engaña al usuario visualmente: en pantalla la lista aparece perfectamente ordenada por ID, pero a nivel de arquitectura de memoria, `empleadosRegistrados` sigue intacto.

5. Conclusiones Técnicas

A nivel matemático, esta implementación de Quick Sort nos garantiza una complejidad temporal promedio de $O(n \log n)$. Sin embargo, el verdadero logro como futuros ingenieros en informática es la drástica reducción del overhead (sobrecarga) del sistema. Al separar lógicamente la posición visual de los datos de su posición física en la memoria estática, nuestro programa es teóricamente capaz de ordenar miles de registros pesados en fracciones de segundo, demostrando que estructurar correctamente los punteros e índices es tan importante como el algoritmo de ordenamiento en sí.

Punteros para el manejo indirecto con clases y estructuras (P1)

1. Introducción al Sistema

En esta práctica desarrollamos un sistema básico de gestión de registros en C++ que relaciona a los usuarios con sus vehículos. El objetivo principal de este reporte es analizar la arquitectura de memoria del programa actual y, basándonos en los conocimientos adquiridos en este tercer semestre, establecer las bases teóricas para su optimización mediante el uso de punteros y asignación dinámica de memoria, un paso fundamental en la materia de Estructuras de Datos.

2. Análisis de la Arquitectura Actual (Composición vs. Memoria)

Para este programa, diseñamos una relación de composición entre dos clases definidas en nuestro archivo de cabecera Persona.h.

Tenemos la clase Auto, que actúa como un componente interno con los atributos precio y año.

- Tenemos la clase Persona, que además de contener atributos primitivos y de cadena (como nombre, ap, am, género y edad), instancia un objeto de tipo Auto en su interior llamado vehiculo.

Actualmente, en nuestro archivo main.cpp, instanciamos esta estructura utilizando un arreglo estático con la declaración `Persona lista[5];`. Aunque esta implementación cumple su función para un entorno controlado con un menú de 7 opciones, presenta deficiencias críticas a nivel de hardware que podemos resolver utilizando punteros.

3. El Problema de la Memoria Estática en Objetos Compuestos

Al declarar `Persona lista[5];` en tiempo de compilación, el sistema operativo reserva inmediatamente en la pila de llamadas (Stack) el espacio completo para 5 objetos tipo Persona. Dado que cada Persona inicializa sus propios atributos y además llama al constructor de Auto para inicializar el vehiculo en cero, estamos desperdiciando memoria RAM.

Si el usuario inicia el programa y no registra a nadie, el programa sigue ocupando el peso en bytes de 5 personas y 5 autos. Además, nuestro método para quitar elementos (Opción 2) simplemente decrementa la variable `cantidadRegistrada`. Esto es un "borrado lógico"; el objeto sigue existiendo físicamente en la memoria consumiendo recursos.

4. Propuesta de Refactorización: Implementación de Punteros

Para que nuestro programa tenga calidad profesional, debemos migrar del Stack (memoria estática) al Heap (memoria dinámica) mediante el uso de punteros.

A. Arreglos de Punteros

En lugar de un arreglo de objetos pesados, la declaración en nuestro main.cpp debería cambiar a un arreglo de punteros:

```
Persona* lista[5];
```

Con esta simple modificación de sintaxis, la lista inicial solo pesaría lo que pesan 5 direcciones de memoria, sin instanciar ni construir ningún objeto anticipadamente.

B. Instanciación Dinámica (Operador new)

Al usar punteros, la Opción 1 de nuestro menú (Agregar elemento) cambiaría drásticamente. Al momento de capturar los datos, utilizaríamos el operador new para reservar la memoria dinámicamente: `lista[cantidadRegistrada] = new Persona();` Esto asegura que los constructores definidos en Persona.cpp solo se ejecuten cuando realmente necesitemos almacenar información, optimizando el ciclo del procesador.

C. Liberación de Memoria (Operador delete)

Actualmente, nuestro destructor `~Persona()` en el archivo Persona.cpp está vacío. En una arquitectura basada en punteros, la Opción 2 del menú (Quitar elemento) no solo restaría el contador, sino que usaría la instrucción `delete lista[cantidadRegistrada];`. Esto destruiría el objeto físico en la RAM y evitaría las fugas de memoria (Memory Leaks), demostrando un control total sobre el ciclo de vida de nuestra estructura de datos.

5. Conclusiones

La versión actual del programa demuestra un correcto dominio de la Programación Orientada a Objetos, específicamente en el encapsulamiento a través de métodos set y get, y la interacción mediante menús controlados por ciclos do-while. No obstante, como estudiantes de Ciencias de la Informática, comprender las limitaciones de `Persona lista[5];` nos abre la puerta al mundo de los punteros. Integrar direcciones de memoria en el futuro cercano no solo hará que este programa sea más rápido y ligero, sino que sentará las bases para transformarlo de un simple arreglo a una Lista Simplemente Ligada de capacidad infinita.

ADT

Arreglo - Pila, Cola y Lista (P3 a la P5)

P3 - Análisis Lógico de la Pila (Stack)

1. Introducción

En esta práctica desarrollamos un sistema en C++ aplicando Programación Orientada a Objetos para modelar múltiples Estructuras de Datos. El proyecto utiliza polimorfismo mediante una clase abstracta base llamada EstructurasDatos. Este reporte se centrará exclusivamente en analizar la lógica y sintaxis de la clase derivada Pila, la cual opera bajo el principio fundamental de LIFO (Last In, First Out).

2. Declaración y Control de Memoria Estática

Para construir la clase Pila, implementamos memoria estática mediante un arreglo primitivo `int datos[MAX]`, donde la constante de capacidad máxima está definida estáticamente en 5 (`MAX = 5`).

El control lógico absoluto de esta estructura recae sobre la variable de rastreo `tope`. En el archivo de implementación, el constructor inicializa esta variable con la instrucción `tope = -1;`. Este valor negativo es el núcleo lógico inicial: indica matemáticamente que no hay ningún índice válido apuntando al arreglo, previniendo así que el programa lea datos basura de la memoria RAM.

3. Lógica Algorítmica de Operaciones Base

La clase encapsula las operaciones clásicas de una pila y las enlaza a los métodos sobrescritos `agregar()` y `quitar()` definidos por la interfaz base.

Inserción (push): El algoritmo primero valida que no exista un desbordamiento (Overflow) comprobando si `tope == MAX - 1`. Si hay memoria disponible, la operación se realiza en una sola línea de código optimizada: `datos[++tope] = valor;`. El operador de pre-incremento (`++tope`) es vital aquí; le indica al procesador que primero debe sumar 1 al índice actual.

- **Extracción (pop):** Para proteger el programa de un subdesbordamiento (Underflow), el método valida que la pila no esté vacía (`tope == -1`). La extracción utiliza la sintaxis: `datos[tope--]`. A diferencia de la inserción, aquí aplicamos un post-decremento: el programa accede y muestra el valor en el índice actual e, inmediatamente después, resta 1 al `tope`. Esto realiza un "borrado lógico", por lo que no necesitamos sobrescribir la celda con un cero; simplemente la ignoramos en futuras operaciones.

4. Conclusión

- La lógica de esta Pila es sumamente eficiente porque delega todo el peso algorítmico al manejo correcto de los operadores de incremento y

decremento de C++. Al aplicar correctamente el encapsulamiento y la herencia, garantizamos que el puntero simulado (tope) jamás exceda los límites de nuestra memoria estática, resultando en una estructura robusta y libre de errores de segmentación.

P4- Análisis Lógico de la Cola (Queue)

1. Introducción

En esta práctica de Estructuras de Datos, construimos un programa basado en Programación Orientada a Objetos aplicando herencia y polimorfismo mediante la clase abstracta EstructurasDatos. Este reporte se enfoca específicamente en el análisis de la clase derivada ColaConcierto, la cual modela matemáticamente el comportamiento de un TDA tipo FIFO (First In, First Out / Primero en Entrar, Primero en Salir).

2. Variables de Control y Memoria

A diferencia de la pila que solo requiere un rastreador, la lógica de una cola lineal en un arreglo estático (`int boletos[MAX]` con `MAX = 5`) requiere el uso de dos punteros simulados (índices): frente y final.

En la implementación dentro de `EstructurasDatos.cpp`, el constructor inicializa ambas variables en `-1`. Esta es la sintaxis base para representar un estado de "ausencia de datos", evitando que el programa intente leer basura de la memoria RAM. La cola se considera completamente vacía matemáticamente cuando `frente == -1`.

3. Lógica de Encolar (Inserción)

El método `formarFan(int numeroBoleto)` maneja la inserción de datos.

Primero, bloquea el desbordamiento (Overflow) si `estaLlena()` devuelve verdadero (cuando `final == MAX - 1`).

- Si es el primer elemento absoluto que entra a la cola (`frente == -1`), el algoritmo debe ajustar el índice `frente = 0` para tener un punto de partida válido para futuras extracciones.
- La sintaxis de inserción es compacta y optimizada: `boletos[++final] = numeroBoleto;`. Mediante el operador de pre-incremento en C++, el puntero `final` se desplaza a la siguiente celda libre y luego se deposita el valor.

4. Lógica de Desencolar (Extracción)

El método `dejarEntrar()` extrae los elementos evaluándolos siempre desde el índice `frente`.

Para evitar un subdesbordamiento (Underflow), aborta la operación si la cola ya está vacía.

Borrado Lógico: El elemento no se elimina de la memoria física del arreglo. En su lugar, aplicamos la instrucción `frente++`. Al mover el índice de inicio hacia adelante, el dato anterior queda lógicamente inaccesible y "eliminado" del flujo.

- **Reseteo del Arreglo:** La parte más crítica de esta sintaxis lineal es la condición `if (frente >= final)`. Si extrajimos al último fanático de la cola, la estructura quedaría inutilizable porque los índices estarían al límite. Por ello, reiniciamos el ciclo volviendo a asignar `-1` a `frente` y `final`.

5. Conclusión

La sintaxis de la clase `ColaConcierto` demuestra un manejo sólido de los índices de arreglos en C++ para simular el comportamiento FIFO. Aunque la implementación lineal mediante borrado lógico (`frente++`) funciona perfecto para este límite de 5 elementos, como área de mejora para el semestre, podríamos evolucionar esta misma lógica hacia una "Cola Circular" usando el operador módulo (%), lo que nos permitiría reutilizar los espacios de memoria que van quedando vacíos al principio sin tener que esperar a que la estructura se vacíe por completo.

P5- Análisis Lógico de la Lista

1. Introducción

En esta práctica de Estructuras de Datos, aplicamos los conceptos de herencia y polimorfismo derivando clases de la estructura abstracta `EstructurasDatos`. Este reporte se enfoca en el análisis de la clase `Lista`, un Tipo de Dato Abstracto (TDA) que, a diferencia de las pilas y colas, permite la inserción secuencial y la extracción aleatoria mediante índices.

2. Control de Memoria Estática

La arquitectura de nuestra lista se basa en memoria estática. En la definición de la clase, declaramos un arreglo `int elementos[MAXIMO]` con una constante `MAXIMO = 10`.

El control lógico de este arreglo recae exclusivamente en la variable `cantidad`. En el constructor, inicializamos `cantidad = 0`;. Esta variable tiene un doble propósito fundamental:

Actúa como contador total de elementos para los métodos `tamano()`, `estaVacia()` y `estaLlena()`.

1. Funciona como el índice lógico ("puntero") que nos indica exactamente en qué celda de memoria debemos insertar el próximo dato.

3. Sintaxis de Inserción Secuencial

El método `insertar(int valor)` maneja la entrada de datos. Tras validar que el arreglo no esté lleno, la inserción se resuelve en una sola instrucción optimizada: `elementos[cantidad++] = valor;` Al usar el operador de post-incremento (`cantidad++`), el programa primero guarda el valor en el índice actual e inmediatamente después le suma 1 a la variable, dejándola lista para la siguiente inserción sin riesgo de sobrescribir datos.

4. Lógica de Eliminación y Corrimiento de Memoria

El método `eliminar(int posicion)` es el bloque algorítmico más complejo de esta clase, ya que extraer un elemento del medio de un arreglo estático genera un "hueco" en la memoria. Para solucionarlo, implementamos un algoritmo de corrimiento (desplazamiento):

- Primero, se valida que el índice solicitado por el usuario exista lógicamente (`posicion >= 0` y `< cantidad`).

Posteriormente, un ciclo `for` itera desde la posición a borrar hasta el penúltimo elemento válido (`cantidad - 1`).

La instrucción `elementos[i] = elementos[i + 1];` mueve físicamente cada dato una celda hacia la izquierda, aplastando el valor que queríamos eliminar y cerrando la brecha de memoria.

- Finalmente, se aplica un borrado lógico general restando uno al contador: `cantidad--;`.

5. Conclusión

La sintaxis empleada en la clase `Lista` nos demuestra las ventajas y desventajas de los arreglos estáticos. Mientras que la inserción es inmediata y tiene una complejidad temporal de $O(1)$ constante, la eliminación aleatoria requiere mover los datos en cascada mediante un ciclo `for`, lo que nos da una complejidad de $O(n)$ en el peor de los casos. Para proyectos futuros más robustos, esta lógica nos justifica la necesidad de migrar hacia Listas Simplemente Ligadas utilizando punteros y memoria dinámica.

Puntero - Pila, Cola y Lista (P6 a la P7Bis)

P6- Lógica y Sintaxis de la Pila

1. Introducción

En esta práctica, dimos el salto fundamental de la memoria estática a la memoria dinámica (Heap). Utilizando Programación Orientada a Objetos, implementamos una clase Pila que hereda de la interfaz abstracta EstructurasDatos. El objetivo de este reporte es analizar cómo la sintaxis de punteros en C++ nos permite modelar el comportamiento LIFO (Last In, First Out) sin las limitantes de un tamaño máximo predefinido.

2. Arquitectura del Nodo y Memoria

Para abandonar el uso de arreglos estáticos, definimos un struct Nodo privado dentro de la clase Pila.

El control absoluto de la estructura se maneja con un único puntero rastreador: `Nodo* tope`.

En el constructor de la clase, inicializamos `tope = nullptr`. Esta es la condición sintáctica absoluta que le indica al programa que la estructura está matemáticamente vacía.

- Una ventaja crítica de esta implementación es visible en el método `estaLlena()`. A diferencia de las pilas basadas en arreglos, aquí el método simplemente retorna `false` de manera incondicional. Al utilizar memoria dinámica, la pila solo se llena si el sistema operativo se queda físicamente sin memoria RAM.

3. Lógica de Inserción (apilar)

`Nodo* nuNodo = new Nodo();` solicita dinámicamente un bloque de memoria para instanciar el nuevo registro.

`nuNodo->dato = valor;` asigna la carga útil ingresada por el usuario.

`nuNodo->sig = tope;` enlaza el nuevo nodo con el resto de la pila, empujando lógicamente a los elementos anteriores hacia abajo.

- `tope = nuNodo;` actualiza nuestro rastreador principal para que ahora apunte a este nuevo elemento superior.

4. Lógica de Extracción y Prevención de Fugas (desapilar)

El método `desapilar(int& valor)` extrae datos por referencia y maneja la recolección de basura manual, un paso obligatorio en C++.

Tras validar que la pila no esté vacía, creamos un puntero auxiliar: `Nodo* ex = tope;`. Esto "sostiene" el nodo superior para no perder su dirección de memoria.

Extraemos el dato y reconectamos el `tope` al siguiente elemento válido: `tope = ex->sig;`.

- La instrucción más importante de este método es `delete ex;`. Esto destruye físicamente el nodo extraído, devolviendo los bytes al sistema operativo y previniendo fugas de memoria (Memory Leaks).

5. Conclusión

La implementación de la Pila Dinámica demuestra el verdadero poder de los punteros en C++. La lógica descrita no solo optimiza el rendimiento general, sino que hace a la estructura completamente autónoma. Esta misma lógica de limpieza la aplicamos en el destructor `~Pila()` iterando un ciclo `while` con `desapilar`, garantizando que al cerrarse el programa, la memoria quede impecable.

P7- Lógica y Sintaxis de la Cola

1. Introducción

En esta práctica de Estructuras de Datos, continuamos nuestro estudio de la memoria dinámica (Heap) implementando una Cola basada en punteros. Esta clase hereda de la interfaz abstracta `EstructurasDatos`. El objetivo de este breve reporte es analizar cómo la sintaxis de C++ nos permite modelar perfectamente el comportamiento FIFO (First In, First Out) sin las restricciones de capacidad que imponen los arreglos estáticos.

2. Arquitectura de Nodos y Doble Puntero

A diferencia de la pila dinámica que solo requiere un rastreador superior, la cola necesita monitorear ambos extremos para mantener la complejidad algorítmica en $O(1)$. En nuestra cabecera definimos un struct `Nodo` y declaramos dos punteros clave: `prim` (frente) y `fin` (final).

En el constructor, ambos se inicializan en estado nulo (`nullptr`). Matemáticamente, el programa sabe que la estructura está vacía simplemente evaluando si `prim == nullptr`. Asimismo, al trabajar con memoria dinámica, el método `estaLlena()` siempre retorna `false`, ya que la única limitante real es la memoria RAM del equipo.

3. Lógica de Inserción (insertar)

Para ingresar un nuevo elemento, el método solicita un bloque de memoria usando `new Nodo()`. La genialidad de esta sintaxis radica en su bifurcación lógica:

- Si la cola está vacía, el nuevo nodo asume el rol de ser el primer elemento absoluto: `prim = NuNodo;`

Si ya existen elementos formados, encadenamos el nuevo nodo al final de la fila apuntando desde el último elemento válido: `fin->sig = NuNodo;`

- Sin importar cuál de los dos casos ocurra, la función siempre termina actualizando nuestro rastreador trasero: `fin = NuNodo;`

4. Lógica de Extracción y Limpieza (retirar)

El método de extracción asegura el cumplimiento de la política FIFO operando exclusivamente sobre el puntero prim.

Tras guardar el dato por referencia, creamos un puntero auxiliar de retención: `Nodo* nEliminar = prim;`. Esto es vital para no perder la dirección de memoria al mover los enlaces.

Manejo del último nodo: Si resulta que `prim == fin`, significa que estamos extrayendo el único elemento que quedaba. El algoritmo resetea la cola devolviendo ambos punteros a `nullptr`. De lo contrario, simplemente desplaza el inicio al siguiente en la fila: `prim = prim->sig;`.

- Finalmente, ejecutamos `delete nEliminar;` para destruir el nodo físicamente y devolverle esa memoria al sistema operativo.

5. Conclusión

La sintaxis implementada en la Cola Dinámica nos enseña a manipular referencias de memoria cruzadas (`sig`) y a gestionar múltiples rastreadores simultáneamente. Al asegurar el uso estricto de la instrucción `delete` tanto en la extracción como en el destructor de la clase (donde un ciclo `while` vacía la cola por completo), hemos construido un Tipo de Dato Abstracto profesional, rápido y libre de fugas de memoria (Memory Leaks).

P7BIS - Lógica y Sintaxis de la Lista

1. Introducción

En esta práctica de Estructuras de Datos, desarrollamos una Lista Dinámica implementando el uso de punteros y gestión de memoria en el Heap. La clase `Lista` hereda de nuestra interfaz abstracta `EstructurasDatos`. Este breve reporte analiza cómo la sintaxis de C++ nos permite enlazar, recorrer y eliminar nodos de forma secuencial sin las restricciones de capacidad impuestas por un arreglo estático.

2. Arquitectura del Nodo y el Puntero Inicial

Para gestionar la memoria, dentro de la definición de nuestra clase `Lista` declaramos un `struct` `Nodo` privado que contiene la variable entera `dato` y un puntero de autorreferencia `Nodo* sig`.

Toda la estructura está anclada a un único puntero rastreador principal llamado `cabeza`. En el constructor del archivo de implementación, inicializamos `cabeza = nullptr;`. Esta sintaxis es el pilar lógico que le indica al programa que la lista está matemáticamente vacía desde el momento en que se instancia.

3. Lógica de Inserción (insertarFin)

A diferencia de la pila que apila datos al inicio, nuestra lista está diseñada para anexar los nuevos elementos al final de la cadena.

Para lograr esto, creamos un nuevo bloque de memoria con `Nodo* NuNodo = new Nodo();` y le asignamos su valor.

Si la estructura está completamente vacía, la inserción es directa: `cabeza = NuNodo;`.

Si la lista ya contiene datos, aplicamos un recorrido lógico. Instanciamos un puntero auxiliar `Nodo* actual = cabeza;` y utilizamos un ciclo `while (actual->sig != nullptr)`. Este ciclo avanza de nodo en nodo hasta detectar el final de la cadena. Al detenerse en el último nodo válido, lo enlazamos al recién creado con la instrucción `actual->sig = NuNodo;`.

4. Lógica de Búsqueda y Extracción

(eliminarEspecifico)

El método de eliminación es el más complejo a nivel sintáctico porque opera basándose en la coincidencia de un valor, no en índices fijos.

Caso Base (Inicio): Si el dato buscado está en la primera posición (`cabeza->dato == n`), creamos un puntero auxiliar de respaldo (`ex`), adelantamos la cabeza al segundo elemento con `cabeza = cabeza->sig;`, y aplicamos `delete ex;` para destruir el elemento inicial.

Caso General (Medio o Final): Si el dato está más adelante, usamos un ciclo que "mira al futuro" evaluando el dato del nodo contiguo: `while (actual->sig != nullptr && actual->sig->dato != n)`.

- Al detenernos exactamente un nodo antes del objetivo, "puenteamos" la conexión para desenlazarlo del flujo lógico de la lista (`actual->sig = ex->sig;`) y procedemos a destruirlo físicamente de la memoria RAM con `delete ex;`.

5. Conclusión

La sintaxis de la Lista Dinámica nos demuestra la flexibilidad del paradigma de nodos entrelazados en C++. Dominar la instrucción `actual = actual->sig` para iterar elementos es un paso crítico en nuestra formación profesional. Asimismo, asegurarnos de utilizar `delete` durante las eliminaciones manuales y dentro del destructor de la clase (el cual vacía la lista por completo al cerrar el programa), nos garantiza crear sistemas totalmente libres de fugas de memoria (Memory Leaks).

Librerías - Pila, Cola y Lista (P8 a la P10)

P8 - Lógica y Sintaxis de la Pila

1. Introducción

En esta práctica dimos un paso muy importante en la materia de Estructuras de Datos al dejar atrás la gestión manual de memoria y comenzar a utilizar la Standard Template Library (STL) de C++. Este breve reporte se enfoca en analizar la clase Navegador, la cual simula el historial de navegación web empleando una Pila nativa bajo el principio LIFO (Last In, First Out).

2. Declaración y Abstracción de Memoria

Para tener acceso a esta estructura, incluimos la directiva `#include` en nuestro archivo de cabecera. Dentro de la definición de la clase Navegador, la estructura se declara simplemente como `stack<string> pilaHistorial;`.

A nivel de tercer semestre, esto representa una abstracción enorme: la STL se encarga por completo de crear los nodos subyacentes y gestionar la memoria dinámica en el Heap. Ya no necesitamos preocuparnos por asignar memoria con `new` ni liberarla con `delete`, evitando así posibles fugas de memoria.

3. Lógica Algorítmica: Inserción y Extracción

La sintaxis para manipular nuestra pila se simplifica utilizando los métodos nativos de la librería en el archivo de implementación:

Inserción (push): Cuando el usuario visita una nueva página, el método `Pagina` encapsula la instrucción `pilaHistorial.push(dir);`. Esto empuja la URL (cadena de texto) automáticamente a la cima de la estructura.

Consulta y Extracción (pop y top): Para simular el botón de "Atrás" en el navegador, el método `Anterior` primero utiliza `pilaHistorial.empty()` para proteger el programa de un Underflow (intentar borrar algo que no existe). Si hay historial, consultamos el valor superior con `pilaHistorial.top()` y procedemos a eliminarlo definitivamente de la memoria ejecutando `pilaHistorial.pop();`.

4. Visualización y el Problema de la Iteración

A diferencia de los arreglos o las listas, la clase `stack` de la STL es muy estricta y no proporciona iteradores; es decir, no podemos usar un ciclo `for` para leer todos los datos desde el tope hasta el fondo.

Para resolver nuestro método `mostrar()` sin perder datos, aplicamos una solución algorítmica específica: instanciamos una pila auxiliar haciendo una copia directa de la original mediante `stack copiaHistorial = pilaHistorial;`. Posteriormente, usamos un ciclo `while` para extraer (`.pop()`) e imprimir (`.top()`) todos los elementos de la pila temporal, dejando nuestro historial real intacto para seguir operando.

5. Conclusión

Implementar una pila mediante la librería nos demuestra cómo se desarrolla el software en entornos profesionales. Aunque haber aprendido a enlazar punteros manualmente fue vital para nuestra lógica matemática, utilizar la STL reduce drásticamente las líneas de código, elimina los errores de segmentación y nos permite centrarnos al 100% en la arquitectura y reglas de negocio de nuestra aplicación (como en este caso, el comportamiento del historial de navegación).

P9 - Lógica y Sintaxis de la Cola

1. Introducción

En esta práctica dimos un avance muy importante en la materia de Estructuras de Datos al sustituir la gestión manual de punteros por la Standard Template Library (STL) de C++. Este breve reporte analiza específicamente la clase Turnos, la cual modela un sistema de atención a clientes apoyándose en una estructura de Cola bajo el principio FIFO (First In, First Out).

2. Declaración y Abstracción

Para poder hacer uso de la cola nativa de C++, agregamos la directiva `#include` en nuestro archivo de cabecera `EstructurasConLibrerias.h`. En esa misma clase, instanciamos nuestra estructura con la instrucción `queue filaC;`.

Como estudiantes de tercer semestre, esto representa una abstracción valiosa: la STL se encarga en segundo plano de gestionar los punteros de frente y final, así como de solicitar la memoria dinámica mediante nodos. Nuestra única preocupación ahora es la lógica del negocio.

3. Lógica Algorítmica: Inserción y Extracción

La manipulación de los datos se realiza en el archivo `EstructurasConLibrerias.cpp` utilizando los métodos nativos, lo que reduce la posibilidad de cometer errores de memoria:

Encolar (push): En el método `registrarC`, usamos `filaC.push(nombre);`. Esto automáticamente reserva el espacio e inserta al nuevo cliente al final de la estructura, operando en tiempo constante $O(1)$.

- Consultar y Desencolar (front y pop): El método `atenderSig` controla la salida. Para evitar un Underflow, primero valida la existencia de datos con `filaC.empty()`. Si la cola no está vacía, extraemos el valor delantero con `filaC.front()` y lo destruimos de la memoria llamando a `filaC.pop();`.

4. El Reto de la Iteración (mostrar)

Una característica estricta de queue en la STL es que no posee iteradores; no podemos usar un ciclo for tradicional para imprimir a los clientes porque romperíamos la regla matemática del encapsulamiento FIFO.

Para solventar esto en nuestro método mostrar, aplicamos una estrategia de clonación: instanciamos una cola temporal copiando la original mediante `queue<string> copiaFila = filaC`, Posteriormente, usamos un ciclo while (`copiaFila.empty()`) para leer el frente e ir destruyendo (`.pop()`) los elementos de la copia. Así logramos imprimir toda la fila en pantalla sin alterar los datos de la cola real con la que estamos trabajando.

5. Conclusión

El uso de `<queue>` simplifica drásticamente nuestro código. Evitamos el manejo explícito de variables temporales, condicionales complejas o las sentencias `new` y `delete`. Esto nos permite programar de manera más segura y con estándares industriales, garantizando un código limpio y libre de fugas de memoria.

P10 - Lógica y Sintaxis de la Lista

1. Introducción

En esta práctica consolidamos nuestros conocimientos de Estructuras de Datos dando el salto a la Standard Template Library (STL) de C++. Este breve reporte se enfoca en el análisis de la clase `Inventario`, la cual gestiona objetos de tipo `Producto` utilizando la estructura dinámica `std::list`.

2. Declaración y Abstracción de Memoria

Para implementar esta estructura, primero agregamos la directiva `#include <list>` en el archivo de cabecera. Al declarar la variable privada `list obInv`, la STL abstrae por completo la gestión de la memoria dinámica. Como estudiantes de tercer semestre, sabemos que internamente esto opera como una Lista Doblemente Ligada, pero la librería nos ahorra la tarea de crear manualmente estructuras `Nodo` y de gestionar los punteros entrelazados, evitando así el riesgo de errores de segmentación.

3. Lógica de Inserción y Extracción

En el archivo de implementación, la manipulación de datos se resuelve de forma directa y segura con los métodos nativos del contenedor:

1. Inserción (`push_back`): El método `agregar()` recopila el código, nombre y cantidad, instancia un objeto `Producto` y lo inserta directamente al final de la

estructura usando `oblInv.push_back(...)`. El sistema operativo maneja la asignación de memoria en tiempo $O(1)$.

- Extracción (`pop_back` y `back`): Para la función `quitar()`, protegemos el flujo verificando primero `!oblInv.empty()`. Para avisar qué producto se eliminará, accedemos al atributo del último elemento con `oblInv.back().nombre` y, en la línea siguiente, lo destruimos de la memoria llamando a `oblInv.pop_back()`.

4. El Uso Avanzado de Iteradores (mostrar)

La gran ventaja de la clase `list` frente a `stack` o `queue` de la STL es que sí permite recorridos secuenciales sin destruir los datos. Para nuestro método `mostrar()`, implementamos un iterador nativo en lugar de copias temporales.

La sintaxis del ciclo `for` es clave aquí:

```
for(list<producto>::iterator it = oblInv.begin(); it != oblInv.end(); ++it)
```

Declaramos un "puntero inteligente" (`it`) que arranca en la primera dirección válida (`begin()`) y avanza lógicamente nodo por nodo (`++it`) hasta que alcanza la marca del final (`end()`). Para invocar la información dentro de cada nodo, utilizamos el operador flecha: `it->mostrarDet()`.

5. Conclusión

Sustituir nuestras listas dinámicas manuales por la librería `<list>` marca un punto de inflexión en nuestra formación. Nos libera de gestionar instrucciones `new` y `delete`, y nos ofrece herramientas poderosas y seguras como los iteradores. Esto nos permite centrarnos completamente en la lógica de negocio —en este caso, un inventario de productos— generando un código más limpio, fácil de leer y libre de fugas de memoria.

ADT - Nuevo tipo de dato

Arreglo - Pila, Cola y Lista (P3 a la P5)

P3.1-Implementación de Pila (Stack)

1. Objetivo de la Práctica

Fue desarrollar y comprender a fondo el funcionamiento de un Tipo de Dato Abstracto (TDA) Pila utilizando memoria estática (arreglos). El objetivo principal es encapsular la lógica de inserción y extracción respetando estrictamente la política LIFO (Last In, First Out).

2. Análisis de la Estructura Interna (La Arquitectura)

A nivel de código, la arquitectura se divide limpiamente en el archivo de cabecera (Pila.h) y la implementación (Pila.cpp). Al estar usando arreglos en lugar de nodos dinámicos con punteros, la estructura de la Pila depende completamente de dos atributos privados dentro de nuestra clase:

- **El arreglo principal:** Un bloque de memoria de tamaño predefinido (por ejemplo, MAX = 100) que guarda nuestros datos.
- **El apuntador lógico tope (top):** Esta es la variable más crítica de la práctica. Es simplemente un número entero que inicializamos en -1 al momento de instanciar la Pila. Su único trabajo es decirnos en qué índice exacto del arreglo vive el último elemento que entró.

3. Retos y Soluciones durante el Desarrollo El mayor reto lógico al construir el main.cpp fue lidiar con los límites de la memoria. Si por accidente intentábamos hacer un Pop de más, el programa intentaba acceder al índice -1 del arreglo, leyendo memoria basura o provocando que el programa colapsara. Aplicar el encapsulamiento correcto nos obligó a validar siempre el estado del arreglo antes de cualquier operación, asegurando que la interfaz de la Pila fuera robusta y a prueba de fallos humanos.

4. Conclusión Desarrollar esta Pila estática es un excelente ejercicio de abstracción. Aunque trabajar con arreglos nos limita a un tamaño máximo fijo, nos enseña que las Estructuras de Datos son esencialmente la combinación de variables de almacenamiento con reglas muy estrictas de acceso. Al encapsular esta lógica, logramos que el usuario final del programa solo vea una "Pila", sin importarle el arreglo o los índices de fondo

P4.1 -Implementación de Cola

1. Descripción del Programa

El proyecto es una simulación para gestionar la fila de atención de un cine. El objetivo principal es modelar la llegada y atención de los clientes utilizando una estructura de datos dinámica de tipo **Cola (Queue)**. Para mantener las buenas prácticas de la Programación Orientada a Objetos (POO), el programa está modularizado en cinco archivos principales: el programa ejecutable (main.cpp), la clase que define los atributos de la entidad (Cine.h y Cine.cpp), y la clase que administra la estructura de datos (ColaCine.h y ColaCine.cpp).

2. Lógica de la Estructura (Cola)

La estructura principal opera bajo el principio **FIFO (First In, First Out)**; es decir, el primer cliente que llega a la taquilla del cine es el primero en ser atendido y, por ende, el primero en salir de la fila.

Para que esto funcione de forma dinámica sin un tamaño fijo predeterminado, utilizamos nodos enlazados y dos apuntadores fundamentales:

- **frente (front):** Apunta al nodo del cliente que está en primer lugar, a punto de ser atendido.
- **final (rear):** Apunta al último nodo (el último cliente) que ingresó a la fila.

3. Sintaxis y Operaciones Principales (ColaCine.cpp)

La lógica interna de nuestra clase se basa en dos operaciones críticas:

- **Encolar (Insertar):** Cuando llega un nuevo cliente, instanciamos un nuevo nodo en memoria dinámica usando la palabra reservada `new`. Si la cola está completamente vacía (cuando `frente == NULL`), tanto el apuntador `frente` como el `final` deben apuntar a este nuevo nodo. Si ya hay clientes formados, hacemos que el puntero siguiente del actual nodo `final` apunte al nuevo nodo, y luego actualizamos nuestro apuntador `final` para que sea este último en llegar.
- **Desencolar (Atender/Eliminar):** Para atender a un cliente, primero extraemos la información del nodo apuntado por `frente`. Después, movemos el apuntador `frente` al nodo que le sigue (`frente = frente->siguiente`). Es crucial liberar la memoria del nodo que acabamos de atender usando `delete` para evitar fugas de memoria (*memory leaks*). Además, validamos mediante un condicional `if` que, si al avanzar el `frente` nos damos cuenta de que quedó apuntando a `NULL` (es decir, la fila se vació por completo), también debemos igualar el apuntador `final` a `NULL`.

4. Conclusión La separación del código en archivos de cabecera (.h) y de implementación (.cpp) nos permite mantener un diseño limpio y respetar el encapsulamiento de los datos del cine. El reto principal de esta práctica no es solo la sintaxis del lenguaje, sino la lógica para asegurar que los apuntadores se actualicen en el orden correcto al insertar y eliminar nodos de la cola, garantizando la integridad de la memoria en todo momento.

P5.1 -Implementación de Lista

1. Descripción del Programa

Este programa fue realizado mediante la estructura de una lista. A diferencia de los arreglos con tamaño estático, este programa nos permite alojar memoria en tiempo de ejecución según se vaya necesitando. Para mantener el código limpio y respetar la POO, el proyecto está modularizado en tres archivos principales: el ejecutable `main.cpp`, y la definición e implementación de la clase en `Nodo.h` y `Nodo.cpp`.

2. Lógica de la Estructura

La lógica central de la lista radica en el uso de apuntadores para conectar elementos aislados en la memoria. Cada elemento es un objeto instanciado de la clase `Nodo`, el cual está dividido en dos partes fundamentales:

- **El campo de dato:** La variable que almacena la información que nos interesa.
- **El apuntador (siguiente):** Un puntero que guarda la dirección de memoria del próximo nodo en la secuencia.

3. Sintaxis y Operaciones Principales (`Nodo.cpp`)

El manejo de la lista depende de la correcta asignación y liberación de memoria:

- **Inserción (Crear Nodos):** Para agregar un elemento, utilizamos el operador `new` para reservar espacio en el *heap* (memoria dinámica). Si la lista está vacía (`inicio == NULL`), el nuevo nodo se convierte en el inicio. Si ya existen elementos, usamos un puntero auxiliar para recorrer la lista mediante un ciclo `while` (hasta que `aux->siguiente == NULL`) y así enlazar el último nodo con el recién creado.
- **Eliminación (Liberar Nodos):** Al borrar un nodo, es vital reasignar los apuntadores para no romper la cadena. El puntero siguiente del nodo anterior debe saltarse el nodo a borrar y apuntar al que le sigue. Inmediatamente después, utilizamos la instrucción `delete` sobre el nodo extraído para destruir el objeto y evitar fugas de memoria (*memory leaks*).

4. Conclusión Separar la clase en su archivo de cabecera (`.h`) y su lógica (`.cpp`) nos permite aplicar correctamente el encapsulamiento. El principal desafío de esta práctica es dominar la sintaxis de los apuntadores (`->`) y asegurar que el orden de asignación al insertar o eliminar sea el correcto; de lo contrario, corremos el riesgo de apuntar a `NULL` por error y provocar que el programa colapse.

Puntero – Pila, Cola y Lista (P6 a la P7Bis)

P6.1 -Implementación de Pila

1. Descripción General del Programa

El objetivo principal del programa es simular el comportamiento de una pila bajo el principio fundamental **LIFO** (*Last In, First Out* - Último en entrar, primero en salir). El código no es un solo bloque, sino que está correctamente modularizado aplicando los principios de la Programación Orientada a Objetos, dividiéndose en tres archivos principales: `Pila.h`, `Pila.cpp` y `main.cpp`.

2. Análisis de la Lógica de la Pila

A nivel lógico, la pila funciona gestionando el acceso a los datos únicamente por un extremo, al que llamamos "tope" (o *top*). La lógica se centró en implementar las operaciones base para que la estructura funcione correctamente:

- **Push (Insertar):** Al agregar un nuevo elemento, se crea un nuevo nodo en memoria. La lógica dicta que el apuntador siguiente de este nuevo nodo debe apuntar a lo que actualmente es nuestro tope, y luego actualizamos el tope para que sea este nuevo nodo.
- **Pop (Extraer):** Para sacar un dato, primero validamos que la pila no esté vacía. Si hay elementos, guardamos el valor del tope actual, movemos el apuntador del tope al *siguiente* nodo, y liberamos la memoria del nodo extraído para evitar fugas de memoria (*memory leaks*).

3. Análisis de Sintaxis y Modularización

La sintaxis del programa refleja una correcta abstracción de datos:

- **En Pila.h:** Se declaran las clases. Aquí definimos la estructura del Nodo (que contiene el dato y el puntero al siguiente nodo) y la clase Pila con sus métodos públicos y atributos privados. Esto nos ayuda a mantener el encapsulamiento.
- **En Pila.cpp:** Desarrollamos la lógica de los métodos declarados en el *header*. Aquí es donde la sintaxis de punteros brilla y se vuelve crítica. Utilizamos el operador *new* para la asignación dinámica de memoria durante el *Push*, y el operador *delete* para liberar la memoria dinámicamente durante el *Pop*.
- **En main.cpp:** Es el punto de entrada. Aquí simplemente instanciamos un objeto de la clase Pila y mandamos a llamar sus métodos para probar su funcionamiento, manteniendo la interfaz de usuario separada de la lógica de la estructura de datos.

4. Conclusión El desarrollo de la práctica, fue un excelente ejercicio para consolidar la sintaxis de punteros y la gestión de memoria dinámica. Implementar la pila desde cero, separando su declaración de su implementación, ayuda a comprender a fondo cómo funcionan internamente las estructuras de datos abstractas antes de depender de librerías estándar.

P7.1 -Implementación de Cola

1. ¿De qué trató el programa?

En este programa utilizamos la implementación de la Cola, a diferencia de la práctica anterior, el objetivo aquí fue cambiar la lógica para que el programa siga el comportamiento **FIFO** (*First In, First Out* - el primero en entrar es el primero en salir). Para mantener las buenas prácticas de la Programación Orientada a Objetos y prepararnos para el examen final de Estructuras de Datos, el proyecto se modularizó en tres archivos distintos: Cola.h, Cola.cpp y main.cpp.

2. Lógica y Sintaxis de la Estructura

- **Declaración y Abstracción (Cola.h):** Aquí se definió la plantilla principal. Declaramos la estructura del Nodo (que guarda el dato y el puntero al siguiente elemento) y la clase Cola con sus métodos públicos y los punteros privados frente y fin.
- **Implementación (Cola.cpp):** En este archivo desarrollamos la lógica fuerte usando punteros y gestión dinámica de memoria.
 - *Encolar (Insertar):* La lógica dicta que las inserciones se hacen por la parte de atrás. Mediante la sintaxis de new, creamos un nodo en memoria. Si la cola está vacía, los punteros frente y fin apuntan a este nuevo nodo. Si ya tiene elementos, hacemos que el puntero siguiente del último nodo apunte al nuevo, y actualizamos el puntero fin.
 - *Desencolar (Extraer):* Las extracciones se hacen por adelante. Primero, la sintaxis valida si hay elementos. Si los hay, creamos un puntero auxiliar que apunte al frente actual, recorremos el puntero frente al nodo siguiente, y finalmente usamos la palabra reservada delete sobre el auxiliar para liberar la memoria correctamente.
- **Ejecución (main.cpp):** Este archivo lo dejamos lo más limpio posible. Solo instanciamos nuestro objeto de tipo Cola y mandamos a llamar los métodos para interactuar con la estructura, separando la interfaz de la lógica interna.

3. Conclusión

El desarrollo de este programa fue clave para terminar de entender cómo se mueven los punteros cuando tenemos dos referencias (frente y fin) al mismo tiempo, a diferencia de otras estructuras más simples.

P7.1Bis -Implementación de Lista

1. ¿De qué trató el programa?

El programa permite insertar nuevos elementos al final de la estructura, buscar y eliminar un dato en específico, y recorrer la lista completa para imprimirla en pantalla.

2. Lógica de la Estructura A diferencia de las pilas (LIFO) o las colas (FIFO), la lógica de esta lista nos permite una manipulación más libre de los datos:

- **Insertión al final (insertarFin):** La lógica aquí es instanciar un nuevo nodo. Si la lista está vacía, nuestro puntero principal (cabeza) apunta directamente a este nuevo nodo. Si ya hay datos, debemos recorrer la lista usando un ciclo while hasta llegar al último nodo (donde el puntero siguiente es nulo) y ahí "enganchar" el nuevo elemento.
- **Eliminación (eliminarEspecifico):** Esta es la parte más compleja a nivel lógico. Primero se valida si el nodo a borrar es el primero de la lista. Si no lo es, se utiliza un puntero auxiliar para buscar el nodo *anterior* al que queremos eliminar. Una vez que lo encontramos, modificamos los apuntadores para "saltarnos" el nodo que vamos a borrar y no romper la cadena de la lista.

3. Sintaxis y Manejo de Memoria

- El código se apoya fuertemente en la sintaxis de punteros (->) para acceder a los atributos de cada estructura (dato y sig).
- Para la gestión de memoria dinámica, utilicé la sintaxis del operador new Nodo() cada vez que se agrega un valor.
- Para evitar las fugas de memoria (*memory leaks*), es indispensable usar el operador delete. Esto se puede ver claramente en el método de eliminar, y sobre todo en el destructor de la clase (~Lista()), el cual implementé con un ciclo while que va borrando nodo por nodo hasta que la lista queda vacía justo antes de terminar la ejecución.

Conclusión Desarrollar esta lista desde cero fue una gran práctica para dominar la sintaxis de los recorridos con punteros (actual = actual->sig). Entender cómo manipular los enlaces entre nodos sin perder la referencia a la memoria es fundamental en la materia, especialmente ahora para el examen final de Estructuras de Datos.

Librerías – Pila, Cola y Lista (P8 a la P10)

P8.1 -Implementación de Pila

1. Objetivo del Programa

El objetivo de esta práctica fue desarrollar un simulador básico del historial de un navegador web. Para lograr esto, implementamos una estructura de datos estática/dinámica tipo **Pila**. El programa simula el comportamiento del botón "Atrás"

que usamos todos los días en internet, donde la última página que visitas es la primera a la que regresas si quieres volver sobre tus pasos.

2. Estructura del Proyecto

Para mantener las buenas prácticas de la programación orientada a objetos en C++, el proyecto desarrollado en el IDE Dev-C++ se dividió modularmente en tres archivos:

- **Navegador.h:** Donde declaramos la clase, la estructura del nodo (que guarda la URL de la página) y los prototipos de los métodos.
- **Navegador.cpp:** Donde desarrollamos toda la lógica y el comportamiento de la pila.
- **main.cpp:** Donde instanciamos el objeto, armamos el menú interactivo para el usuario y mandamos llamar las funciones.

3. Lógica y Sintaxis de la Pila

La esencia de este programa se basa en el principio **LIFO (Last In, First Out)**.

- **Visitar una página (Lógica de Push):** Cada vez que el usuario ingresa una nueva dirección web, creamos un nuevo nodo en memoria. La sintaxis clave aquí es hacer que el enlace (siguiente o next) de este nuevo nodo apunte hacia donde está apuntando actualmente el tope de la pila, y luego actualizar el tope para que sea este nuevo nodo. Así, la nueva página queda "hasta arriba".
- **Regresar a la página anterior (Lógica de Pop):** Cuando el usuario decide retroceder, verificamos primero que la pila no esté vacía (para evitar errores en tiempo de ejecución o caídas del programa). Si hay historial, creamos un puntero auxiliar que apunte al tope actual, movemos el tope al nodo de abajo (tope = tope->siguiente) y, muy importante para no dejar basura en memoria, usamos delete sobre el puntero auxiliar para destruir la página actual de la que acabamos de salir.
- **Mostrar la página actual (Lógica de Top):** Es simplemente una función de consulta. Retorna la información (la URL) del nodo al que apunta el tope en ese momento, sin modificar ni eliminar nada en la estructura.

4. Conclusión

Me pareció un ejercicio excelente para aterrizar la teoría. A veces el concepto de "el último en entrar es el primero en salir" se siente muy abstracto cuando solo lo vemos en el pizarrón, pero al aplicarlo directamente a un caso de uso tan real como el historial de un navegador, la utilidad de las pilas cobra mucho sentido. Es un reto muy grande poder o tener que hacer algo así, pero con la practica fui adquiriendo algo de conocimiento.

P9.1 -Implementación de Cola

1. Objetivo del Programa

El objetivo de esta práctica (P9.1) fue simular el sistema de atención de la recepción de un hotel. Para resolver este problema, implementamos una estructura de datos dinámica tipo **Cola (Queue)**, ya que el comportamiento natural de una fila de espera requiere la lógica **FIFO (First In, First Out)**: el primer huésped en llegar debe ser el primero en recibir su habitación.

2. Estructura del Proyecto

El proyecto fue compilado en el entorno Dev-C++ y, siguiendo las buenas prácticas del paradigma orientado a objetos, lo dividimos en tres archivos:

- **RecepcionHotel.h**: Aquí declaramos la clase, la estructura del nodo (que almacena los datos de cada huésped) y los prototipos de las operaciones de la estructura.
- **RecepcionHotel.cpp**: En este archivo desarrollamos la lógica y el movimiento de los punteros para hacer funcionar la cola de forma dinámica.
- **main.cpp**: Donde instanciamos el objeto y creamos el menú interactivo para que el usuario (el recepcionista) utilice el sistema.

3. Lógica y Sintaxis de la Estructura

A diferencia de una pila, la sintaxis en C++ para una cola requiere que controlemos dos punteros simultáneamente: uno para el **frente** (el huésped que va a ser atendido) y otro para el **final** (donde se forman los nuevos huéspedes). Las operaciones clave fueron:

- **Llegada de un huésped (Lógica de Enqueue)**: Cuando un nuevo huésped llega a la recepción, instanciamos un nodo en memoria. La lógica dicta que si la cola está vacía, ambos punteros (frente y final) deben apuntar a este único nodo. Si ya hay gente en la fila, hacemos que el enlace (siguiente) del nodo final actual apunte al nuevo nodo, y luego actualizamos nuestro puntero final para que sea este último recién llegado.
- **Atender huésped (Lógica de Dequeue)**: Primero validamos que la cola no esté vacía. Luego, usamos un puntero auxiliar para guardar la dirección de memoria del nodo que está en el frente. Movemos el puntero frente al siguiente nodo en la fila (frente = frente->siguiente) y, finalmente, liberamos la memoria del huésped ya atendido usando delete sobre el auxiliar. Así evitamos fugas de memoria en el sistema.

- **Consultar turno:** Es una función de lectura que retorna los datos del nodo al que apunta el frente en ese momento, sin sacarlo de la fila, útil para saber quién sigue.

4. Conclusión

Desarrollar esta recepción de hotel fue un excelente ejercicio para consolidar el manejo de la memoria dinámica. El mayor reto lógico en la sintaxis de C++ fue recordar actualizar correctamente ambos punteros (frente y final) al momento de insertar un nodo, especialmente previendo el caso de cuando la cola se queda vacía después de hacer el *pop* del último huésped.

P10.1 -Implementación de Lista

1. ¿De qué trata el programa? Es un sistema en C++ que gestiona un inventario básico utilizando una estructura de datos de tipo Pila. Permite registrar artículos y retirar el último que se ingresó. Para mantener un buen diseño orientado a objetos, el proyecto se dividió en tres archivos: `main.cpp`, `Articulos.h` y `Articulos.cpp`.

2. Lógica de la Pila

Toda la lógica funciona bajo el principio **LIFO** (*Last In, First Out*). El concepto clave aquí es el control de la "cima" (Top). A diferencia de un arreglo donde puedes acceder a cualquier posición con un ciclo, la lógica de la pila nos restringe a insertar (push) o eliminar (pop) elementos únicamente desde la cima.

3. Sintaxis y Modularidad

Para tener un código limpio, separamos la estructura:

- **Articulos.h:** Contiene la sintaxis de la clase. Aquí declaramos los atributos privados (datos del artículo y el apuntador al siguiente) y los prototipos de las funciones públicas.
- **Articulos.cpp:** Contiene el desarrollo lógico de la clase. Por ejemplo, en la sintaxis de inserción, instanciamos el nodo, enlazamos su apuntador hacia la cima actual, y luego actualizamos la cima para que sea este nuevo nodo.
- **main.cpp:** Es el archivo de ejecución. Su sintaxis se limita a instanciar la pila y mostrar un menú al usuario para mandar a llamar los métodos, sin mezclar la lógica de la estructura de datos.

Conclusión Separar la declaración en el `.h` y la implementación en el `.cpp` hizo que el código fuera mucho más fácil de leer. Implementar la lógica LIFO fue un buen ejercicio para entender cómo restringir el acceso a la información y manejar correctamente la memoria y los apuntadores al momento de apilar y desapilar.